When Does Streaming Tool Use Help?

Characterizing Tool-Intent Stabilization in Streaming Retrieval-Augmented Generation

Elroy Galbraith, PhD
SMG Labs

June 18, 2026

Abstract

Streaming Retrieval-Augmented Generation (Streaming RAG) reduces user-perceived latency by issuing tool queries in parallel with ongoing user input, before the utterance is complete. Reported gains are aggregate, yet the mechanism's benefit is fundamentally *query-intrinsic*: speculation can only help when the correct tool query becomes determinable before the user stops speaking or typing. We isolate and measure this property—**tool-intent stabilization**, the point in the input stream at which a speculative query's retrieval converges to the answer-bearing result. On the CRAG benchmark (1371 validation questions) we (i) measure the distribution of stabilization, (ii) derive a model-agnostic bound H on the portion of tool latency that can be hidden behind the user's remaining input, as a function of tool latency L and input cadence δ, (iii) validate against a working streaming pipeline that realized savings meet or exceed this bound, and (iv) identify which query properties predict early versus late stabilization. The study requires no model training and runs on commodity CPU hardware. We find that sufficiency stabilizes early on the groundable, BM25-retrievable subset of the benchmark (mean $\phi_{\text{suf}} = 0.26$ over the 21.3% of questions where gold evidence is both verbatim-present and retrievable); that on this subset 95.2% of queries admit substantial latency hiding at a realistic operating point (the headline 73.9% that also counts the remainder is dominated by a weaker, top-1-settling notion); and that question type produces a significant but coarse effect on stabilization (Kruskal–Wallis $p = 0.017$, ε^2=0.04)—a robust early/late split rather than a precise per-type order—directly informing when a learned speculative trigger is worth its cost.

1 Introduction

Tool-augmented dialogue systems increasingly rely on external retrieval to remain factual, but tool calls add latency that disrupts conversational flow. Streaming RAG [2] addresses this by predicting and issuing tool queries *in parallel with* the user's still-arriving input, then reflecting on whether retrieved results suffice. The approach reports sizable accuracy and latency improvements and is modality-agnostic, applying equally to typed input.

The reported latency benefit, however, is an aggregate over a benchmark. It leaves a basic question unanswered: **for which queries can speculation help at all, and by how much?** The answer is not a property of the model, the retriever, or the speech stack—it is a property of *where, within an utterance, the information that determines the tool query first appears.* If the decisive term arrives last ("who makes the console called the *PlayStation*"), no early speculative query can retrieve the right evidence and the latency win collapses to zero regardless of system quality. If the decisive term arrives early, almost the entire tool latency can be hidden behind the remaining input. We initially expected these two regimes to show up as a *bimodal* distribution of

stabilization; empirically (§5.1) the picture is better described as a large early-stabilizing mass with a thin late tail—the early case is common and the late case is the rarer failure mode—and we report the distribution as found rather than as hypothesized.

We study this directly. By characterizing tool-intent stabilization as a measurable, query-intrinsic quantity, we obtain (i) a model-agnostic bound on the share of tool latency that can be hidden behind ongoing input, computable *before* any system is built; (ii) a principled account of the failure mode; (iii) actionable guidance on whether a learned trigger is worth its cost; and (iv) results that transfer across text and speech because the quantity is modality-agnostic. Our contribution is measurement and analysis rather than a new system.

2 Background and Related Work

Retrieval-Augmented Generation [10] grounds generation in retrieved evidence; dense passage retrieval [8] and the sparse BM25 model [12] are the standard retrieval backbones. Streaming RAG [2] adds a speculative dimension: a *Trigger* decides when to fire a tool query from a partial utterance, multiple speculative *Threads* run in parallel, and a *Reflector* judges whether results suffice. The idea is closely related in spirit to speculative decoding [9], which hides latency by doing work that may be discarded; here the discarded work is a premature tool call. Our analysis is orthogonal to the system: we quantify the headroom that any such system can exploit, set by the query alone.

Incremental comprehension and anticipation. That the referential intent of an utterance is fixed incrementally—often well before the utterance ends—is among the most robust findings in psycholinguistics. Altmann and Kamide [1] showed that a verb such as "eat" directs anticipatory attention to edible objects before they are named, so partial input already restricts what can follow. Kamide et al. [7] extend this cross-linguistically: in head-final Japanese, case-marked pre-verbal arguments let comprehenders anticipate a forthcoming argument even before the verb (the head) is heard—*pre-head* anticipation [6]. This is the human analogue of firing a tool query from a prefix, and it carries a prediction we return to in H5: *where* the disambiguating cue sits—which varies with word order and case morphology—governs how early intent stabilizes.

Stabilization over a growing prefix. The engineering counterpart—when a prediction over a growing input prefix becomes safe to commit—is the central question of *simultaneous* and *incremental* NLP. Simultaneous machine translation must decide, token by token, whether enough source has arrived to emit output: wait-k and prefix-to-prefix policies [11] and learned read/write agents [5] trade latency against the risk of committing before the disambiguating word arrives—precisely our L-versus-residual tension—and Grissom II et al. [4] cast it as an agent that waits until it can confidently predict the sentence-final verb before committing, whose failure mode—premature commitment forcing revision—is exactly what our volatility V records. Closest to our measurement is incremental spoken language understanding: DeVault et al. [3] build a meaning frame from partial ASR and quantify understanding as a function of prefix length, and the incremental-unit framework [13] formalizes the same prefix-monotonicity and revision concerns. These efforts target a *closed* intent or dialogue-act ontology. Our contribution is to port the lens to *tool/retrieval* intent over an open-world knowledge benchmark: rather than learning a commit policy, we *measure* the query-intrinsic point at which the retrieval target stabilizes, which upper-bounds what any such policy could hide.

3 Problem Formalization

Let a query Q be a word sequence $q_{1:n}$ of length n words, and let $q_{1:t}$ denote the prefix of the first t words (we measure stabilization in words throughout, matching the whitespace tokenization used by both the retriever and the input-cadence model below). Let R be a retriever returning a ranked document list; write $d_t = \text{top-1}(R(q_{1:t}))$ for the top document retrieved from the prefix, d_n for the full-query top document, and $d^\star$ for the gold answer-bearing document.

Self-consistency stabilization t_{sc}. the smallest t such that for all $s \geq t$, $d_s = d_n$: the prefix already retrieves what the full query will.

Sufficiency stabilization t_{suf}. the smallest t such that $d^\star \in \text{top-}k(R(q_{1:t}))$: the prefix already surfaces the gold evidence. This is the operationally meaningful notion, since the full query itself may retrieve imperfectly.

Stabilization fraction $\phi = t^\star/n \in (0, 1]$, reported for both t_{sc} and t_{suf}. Lower ϕ means earlier stabilization and more headroom to hide latency.

Stabilization volatility V, the number of top-1 changes occurring *after* d_n is first reached. High V signals early-commit risk: a confident-looking early result that a later token would overturn.

Hidden latency.

$$H(Q; L, \delta) = \min\Big(L,\ \max\big(0,\ \tfrac{n-t^\star}{\delta}\big)\Big), \tag{1}$$

where L is tool latency (ms) and δ is the input cadence in words per second, so that $(n - t^\star)/\delta$ is the residual input time (in seconds, scaled to ms) still to arrive after the query has stabilized. H is the portion of tool latency that completes behind the user's remaining input. It bounds the *input-hideable* component of the per-query perceived-latency saving—not the total saving, since a real pipeline can additionally overlap other costs (§5.4). A slower cadence (smaller δ) lengthens the residual and so raises H when tool latency is the larger term.

Streamable. Streamable$(Q; L, \delta, \theta)$ holds iff $H \geq \theta L$ for a coverage threshold θ (e.g. $\theta = 0.8$ hides at least 80% of tool latency). The streamable fraction of a workload is the central deployment-relevant statistic.

Hypotheses. The study is organized around four *pre-specified* hypotheses (stated in the proposal that preceded the analysis; we make no claim of a formal public pre-registration), each mapped to a research question (RQ) below.

H1 (RQ1). The distribution of ϕ is bimodal: a large early-stabilizing mass (decisive entity stated up front) plus a tail that stabilizes only near the end (decisive term last).

H2 (RQ2). For large tool latency L, the binding constraint on H is the residual input time $(n - t^\star)/\delta$ rather than L; consequently a *slower* input cadence δ paradoxically *raises* the hideable fraction.

H3 (RQ4). CRAG question type predicts ϕ: simple lookups stabilize early, while comparison, aggregation, multi-hop, and post-processing queries stabilize late.

H4 (robustness). Switching from sparse (BM25) to dense retrieval shifts absolute $t^\star$ but preserves the ordering of ϕ across types.

H5 (cross-linguistic). In verb-final (SOV) languages, queries whose tool intent hinges on the predicate stabilize later in linear word order than in SVO English; but early case morphology can restore earlier stabilization, so the driving cue shifts from word position to morphology, and a case-blind lexical retriever (BM25) stabilizes later on SOV input than a morphology-aware or dense one.

We test H1–H3 directly (§5); the dense-retriever condition (H4) and the cross-linguistic SOV condition (H5)—the latter requiring a natively authored verb-final QA set—are left to future work, and we instead report a top-k robustness sweep (§4, Table 3) as a within-retriever sensitivity check. We report outcomes including disconfirmations: H2 is confirmed, but H1 is *falsified* (the distribution is right-skewed, not bimodal; §5.1) and H3 is contradicted in direction (entity position, not reasoning complexity, governs the ordering; §5.2).

4 Method

Data. We use CRAG, the Comprehensive RAG Benchmark [14], Task 1&2 dev set: 2706 questions, of which 1371 are the validation split we analyze. Each question carries up to five retrieved web pages, a gold answer (plus alternates), and a question-type label.

Grounding $d^\star$. CRAG provides no gold-*passage* label, only gold answer strings. We derive $d^\star$ by grounding the normalized gold answer(s) into passage text: substring match for multi-token answers, and word-boundary match for single-token answers, over the answer and its alternates. Questions whose answer is never a literal span (false-premise, "I don't know", and many aggregation/dynamic answers) are *ungroundable* and excluded from t_{suf}; we report the groundable rate explicitly. Because t_{sc} requires no grounding, we report it as a grounding-free robustness check alongside t_{suf}.

String grounding is a recall-oriented heuristic and can produce false positives: a single-token gold answer (a year, a number, "yes"/"no") may match a passage *coincidentally*, registering sufficiency before the genuinely answer-bearing evidence arrives and biasing t_{suf}—and thus ϕ_{suf}—*early*. Multi-token answers, which require a contiguous substring match, are far less prone to this. We therefore read ϕ_{suf} as a lower bound on stabilization time on the groundable subset, and treat the early-stabilization finding as directionally robust rather than exact.

To bound the population-selection bias this strictness induces, we also run a *relaxed* grounding arm (§5.5): for multi-token answers it accepts a passage if all of the answer's content tokens (stopwords dropped) appear anywhere in it—bag-of-content-token containment, dropping only the contiguity requirement. This is deliberately higher-recall and lower-precision than the substring rule, so the two arms *bracket* the true groundable population and the ϕ_{suf} that goes with it.

Passages and retrieval. Page HTML is cleaned to text and chunked into 120-word windows with a 20-word overlap (stride 100), capped at 400 chunks per question. We index each question's passages with a pure-Python lexical BM25 [12] (k_1=1.5, b=0.75) and retrieve over every prefix $q_{1:t}$, $t = 1 \dots n$, recording d_t and the top-k set at each prefix. We sweep $k \in \{1, 3, 5\}$ for the sufficiency definition. All retrieval is deterministic; the analysis uses the CRAG `dev_v4` snapshot (the released web pages, so retrieval is fixed and not subject to live drift). The two stochastic or selection steps are seeded and reproducible: the 60-question RQ3 subset is simply the first 60 retrieved-gold questions

in dataset order (no sampling), and the bootstrap confidence intervals (§5.2) use 10,000 resamples at seed 0.

Latency model and harness (RQ3). We stream each query as a uniform word sequence—a controlled proxy for input cadence δ—and sweep (L, δ, θ) analytically over the grid $L \in \{100, 300, 600, 1000\}$ ms, $\delta \in \{2, 3, 4\}$ w/s, $\theta \in \{0.5, 0.8, 1.0\}$ to obtain the streamable surface (RQ2). For RQ3 we replay a subset of questions through a faithful asynchronous Streaming RAG harness (fixed-interval Trigger, parallel Threads, Reflector) calibrated with the per-stage latencies reported by Arora et al. [2] (Table 3): query generation ≈ 590 ms and fuse/reflect ≈ 2520 ms, in addition to the swept tool latency L. We compare the *measured* perceived-latency saving (baseline minus streaming) against the H bound.

Reproducibility. The pipeline is training-free and CPU-only. We release the harness code together with the derived $d^\star$ passage labels (the only non-trivial artifact, since CRAG ships none), so that every per-question metric and the full (L, δ, θ) grid can be recomputed offline without re-running retrieval.

5 Results

We analyze the 1371 validation questions of CRAG Task 1&2. Under our string-grounding procedure, 35.4% of questions are *groundable* (the gold answer appears verbatim in the retrieved pages); the remainder—dominated by false-premise items and answers that never occur as a literal span (many aggregation and dynamic questions)—are excluded from sufficiency statistics. At k=3, BM25 surfaces the gold passage in the top-k at some prefix for 21.3% of all questions. All reported ϕ_{suf} statistics are conditional on this retrieved-gold population; ϕ_{sc} is defined for every question and serves as a grounding-free check.

This retrieved-gold subset is not a random sample of the benchmark: it is the intersection of questions whose answer is verbatim-groundable *and* whose gold passage BM25 can surface from some prefix. Both filters plausibly select for queries with a salient, early lexical anchor, so the conditional ϕ_{suf} statistics should be read as characterizing this favorable slice, not "most answerable questions"; if anything, the selection biases the reported stabilization *earlier* than it would be on the full workload. We therefore lean on ϕ_{sc}, defined on all 1371 questions, as the grounding-free companion to every sufficiency claim, and we bound the magnitude of the selection effect with a relaxed-grounding arm below (§5.5): recovering a sixth more of the population moves ϕ_{suf} by under 0.02, so the early-stabilization finding survives the expansion rather than dissolving into it.

5.1 RQ1: How is stabilization distributed?

The two stabilization notions behave very differently (Fig. 1). Self-consistency stabilizes *late*: the lexical top-1 keeps changing until, on average, ϕ_{sc} reaches mean 0.67 (median 0.73)— roughly three-quarters of the way through the query. Sufficiency, in contrast, stabilizes *early*: the gold passage enters the top-k at mean $\phi_{\text{suf}} = 0.26$ and median 0.14. The gap is the central empirical observation of this study: **the answer-bearing document is retrievable from a short prefix long before the query's lexical top-1 settles**. For streaming tool use this is the favorable regime—one need not wait for the utterance to "finish" for the right evidence to be in hand.

The ϕ_{suf} distribution (Fig. 1) is right-skewed with a large mass near zero and a thinner late tail, rather than cleanly bimodal as H1 anticipated. Volatility is modest: the post-stabilization top-1

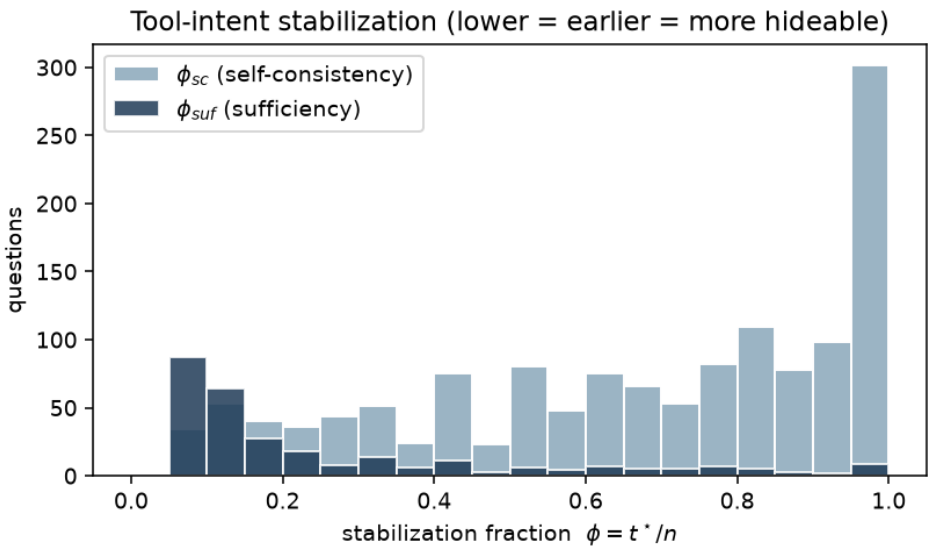

Figure 1: Distribution of stabilization fraction ϕ at k=3. Sufficiency (ϕ_{suf}, dark) concentrates near zero—gold evidence is retrievable early—while self-consistency (ϕ_{sc}, light) is late and broad.

changes at all for only 20.9% of questions (mean $V = 0.52$), so early-commit risk is concentrated in a minority of queries rather than pervasive.

5.2 RQ4: What predicts early vs. late stabilization?

Question type is an ordered predictor of ϕ_{suf}, spanning a roughly threefold range (Table 1, Fig. 2). A Kruskal–Wallis test across all eight types rejects equality of the ϕ_{suf} distributions (H=17.0, df=7, p=0.017); restricting to the five classes with $n \geq 10$—which carry the effect—the test is stronger (H=13.9, df=4, p=0.008). The effect, though significant, is *small*: the rank-based effect size is ε^2=0.04 (both for all eight types and for the five large classes), i.e. question type explains on the order of 4% of the rank variance in ϕ_{suf}. Consistent with a small effect, the bootstrap 95% confidence intervals in Table 1 overlap substantially for adjacent types: the ordering is reliable only at the extremes (*aggregation* and *comparison* earliest, *set* latest), not as a strict cell-by-cell ranking, and the small-support types (`set` n=2, `post-processing` n=5) have intervals too wide to place. A Dunn's post-hoc with Holm correction makes this precise: *no* pairwise contrast survives at α=0.05 (the closest, `simple` vs. `aggregation`, reaches p_{Holm}=0.05). The omnibus effect is therefore carried by the overall rank distribution, not by any single separable pair. We accordingly claim a significant but coarse type-level effect—a robust early/late *tendency*, not a precise per-type order. The qualitative ordering refines hypothesis H3 rather than confirming it. H3 expected simple lookups to stabilize early and aggregation/comparison/multi-hop to stabilize late. We instead find *aggregation* and *comparison* among the *earliest*-stabilizing types, with *set* questions latest. The explanation is that sufficiency measures when the gold *document* becomes retrievable, not when the answer can be *computed*: a comparison or aggregation question often names its key entities up front, so the answer-bearing page is retrievable early even though producing the final answer requires post-retrieval reasoning. In other words, retrieval-sufficiency stabilization is governed by entity position in the utterance, which is not aligned with reasoning complexity. This entity-position account is carried by the well-populated classes (`simple`, `comparison`, `aggregation`, `simple_w_condition`), so it does not hinge on the noisy small-n cells.

Table 1: Sufficiency stabilization by CRAG question type (k=3, retrieved-gold questions), sorted earliest to latest. The last column is a bootstrap 95% CI for the mean (10,000 resamples); intervals overlap heavily for adjacent types, so only the extreme contrasts are reliable.

question type	n	mean ϕ_{suf}	95% CI
aggregation	32	0.198	[0.13, 0.28]
comparison	59	0.210	[0.16, 0.27]
multi-hop	15	0.267	[0.14, 0.42]
simple	118	0.280	[0.24, 0.33]
false_premise	9	0.289	[0.14, 0.48]
simple_w_condition	52	0.307	[0.23, 0.40]
post-processing	5	0.310	[0.10, 0.52]
set	2	0.658	[0.62, 0.70]

5.3 RQ2: What fraction of queries admit latency hiding?

Using $t^\star = t_{\text{suf}}$ where available and falling back to t_{sc} otherwise, we compute the streamable fraction over the full (L, δ, θ) grid (Table 2, Fig. 3). At the central operating point (L=600 ms, δ=3 w/s, θ=0.8), 73.9% of queries admit hiding at least 80% of tool latency.

This blended figure mixes two populations and two notions of stabilization, and we report it decomposed. On the 21.3% of queries with retrieved gold—where $t^\star = t_{\text{suf}}$ reflects evidence actually in hand—95.2% are streamable; on the remaining 78.7%, where we fall back to the grounding-free t_{sc} (top-1 stopped moving), 68.1% are. The sufficiency-only number, 95.2%, is the one that matches our framing and is the honest headline for "evidence is retrievable in time"; the 73.9% blend is lower because it is dominated by the t_{sc} majority, for which "streamable" means only that the lexical top-1 has settled, not that gold is in the top-k. Readers who care about evidence sufficiency should weight the 95.2% figure; those who treat top-1 stability as the deployment signal get the blend.

The blend also counts only upside. "Streamable" asks whether enough tool latency *can* be hidden; it is silent on whether the speculative query was *right*, and H's $\max(0, \cdot)$ floor cannot represent the negative saving a mis-fire incurs (§5.4). One might expect this blind spot to bite hardest in the t_{sc}-fallback majority, which stabilizes *late* (mean ϕ=0.68 vs. $\approx$ 0.29 on the retrieved-gold replay sample), giving a fixed-interval trigger more room to fire before intent settles. We tested this by replaying 60 fallback questions (the harness previously replayed only retrieved-gold) through the same pipeline at L=600 ms. The expectation does *not* hold: the net-negative-saving rate is 1.7% (1/60), identical to the 1.7% on the favorable retrieved-gold population, and mean savings are likewise comparable. The reason is the same constant that dominates RQ3 (§5.4): the query-generation overlap is hidden regardless of whether the speculative *retrieval* was right, so late stabilization lowers H without proportionally raising the mis-fire rate at this operating point. Mis-fires are thus real but rare ($\approx$ 1.7%) across both populations here; the streamable fraction should still be read as an upper envelope on benefit paired with this small measured downside, and the rate may grow at larger L or with a more aggressive trigger—a sweep we leave to future work.

Two further patterns stand out. First, for small tool latency ($L \leq 300$ ms) the constraint is essentially never binding: 82.6% of queries are streamable regardless of cadence—the residual input time trivially covers a short tool call. Second, and confirming **H2**, when tool latency is large (L=1000 ms) the binding constraint becomes residual input time, so a *slower* cadence *raises* the streamable fraction: from 54.5% at δ=4 w/s up to 73.9% at δ=2 w/s. Faster typists and faster talkers leave less room to hide a slow tool.

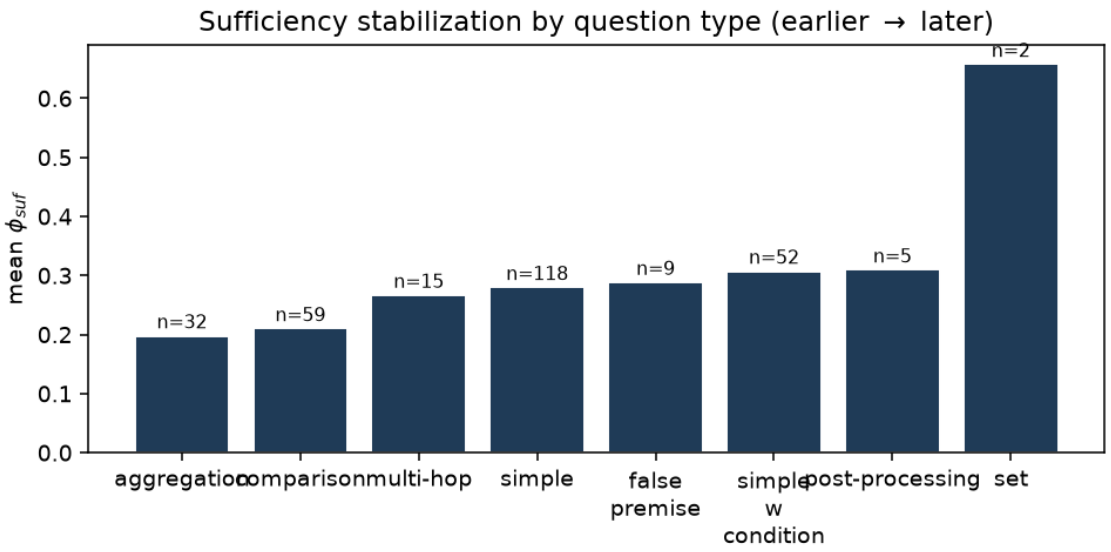

Figure 2: Mean ϕ_{suf} by question type (k=3). Type spans a 3× range, but the ordering tracks entity position, not reasoning complexity.

Table 2: RQ2: streamable fraction (%) at θ=0.8 over the (L, δ) grid. Rows are tool latency L (ms); columns are input cadence δ (words/sec). For large L, slower cadence (lower δ) hides more (H2).

	δ=2	δ=3	δ=4
L=100	82.6	82.6	82.6
L=300	82.6	82.6	82.6
L=600	82.6	73.9	73.9
L=1000	73.9	63.3	54.5

5.4 RQ3: Does the bound match a working pipeline?

We replayed the same 60 retrieved-gold questions through the asynchronous Streaming RAG harness at δ=3 w/s and *two* tool latencies, $L \in \{600, 1000\}$ ms (Fig. 4). In the aggregate the bound is *conservative* at both: measured perceived-latency savings (baseline minus streaming) average 1122 ms vs. the 590 ms H bound at L=600, and 1457 ms vs. 950 ms at L=1000. The overshoot is expected and informative: H models only the tool latency hidden behind input, whereas the calibrated pipeline additionally overlaps the LLM's query-generation step (≈ 590 ms [2]) with the user's input. Indeed the overshoot is almost exactly that step: at L=600 the bound mean (590 ms) plus the query-generation constant (≈ 590 ms) recovers the measured mean (≈ 1122 ms) to within noise—a useful sanity check, but also a caution that this paragraph mostly re-derives H plus a constant rather than independently testing H (we address that next). So a query that H deems streamable will, in practice, save *at least* as much as predicted—the property that makes H safe to plan with.

We are deliberately careful about the stronger claim the bound does *not* support: per-query prediction. At L=600 the bound saturates—59 of the 60 queries share H=600 ms (the input residual exceeds L)—leaving no horizontal spread to test ranking, and the rank correlation between H and the measured saving is negligible (Spearman $\rho = +0.14$). We therefore re-ran the *identical* questions at L=1000 specifically to break the saturation: H now takes three distinct values (0, 667, 1000 ms)

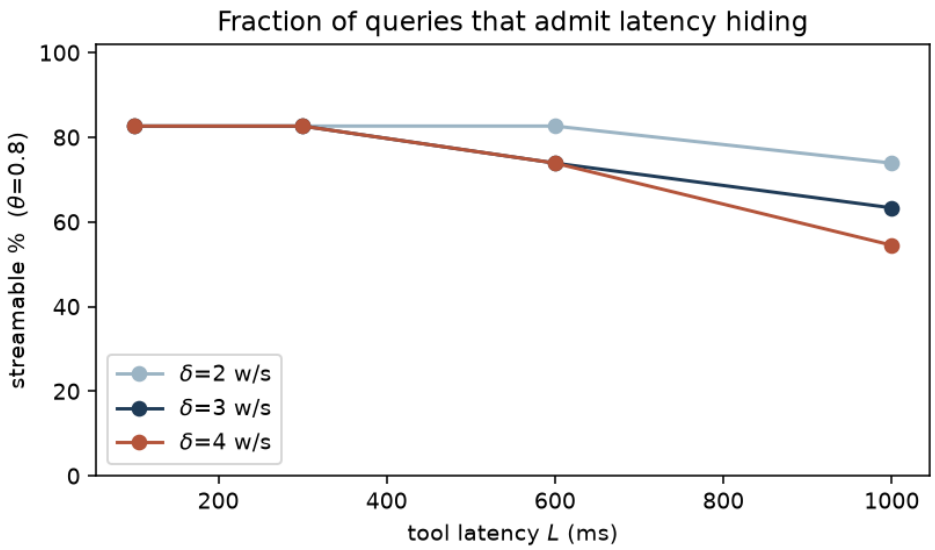

Figure 3: Streamable fraction vs. tool latency L at θ=0.8, one line per cadence δ. The curves separate only once L is large enough for residual input time to bind; there, slower cadence dominates (H2).

with 7 of 60 queries below the cap. The spread appears, but the correlation does not: $\rho = +0.12$ over all 60, and exactly 0.00 within the unsaturated tail. Two structural reasons explain why raising L cannot rescue per-query prediction. First, the dominant component of realized saving—the constant ≈ 590 ms query-generation overlap—is independent of H, so it adds a near-uniform offset that swamps the cross-query signal H encodes. Second, the failure mode is *negative*, and H's $\max(0, \cdot)$ floor cannot represent it. RQ3 thus confirms H as a conservative aggregate bound but *not* a per-query ranker; a predictive model would need to add the query-generation overlap to the hideable budget and replace the floor with a mis-fire penalty.

The negative tail makes the second point concrete and reconnects volatility V to outcomes. The single query that *loses* time—a 17-word comparison ($t^\star$=13)—costs -543 ms at L=600 and -940 ms at L=1000, yet its H is *maximal* (1000 ms): with $n - t^\star$=4 words of nominal residual the bound rates it fully streamable while the fixed-interval trigger, firing before intent settles at word 13, mis-retrieves and pays a re-fire penalty. Notably this query has V=0, so volatility did *not* flag it—the risk here is *late* stabilization under an early-firing trigger, not post-stabilization churn. The actionable lesson is that an early-commit safeguard should gate on ϕ (how late intent settles relative to the trigger cadence) at least as much as on V; validating which signal best predicts mis-fires at scale is the natural next step.

5.5 Robustness

Grounding (bounding the selection bias). Our sharpest claims live on the 21.3% retrieved-gold slice, whose selection plausibly biases stabilization early (§5.1). To bound that bias we re-derive $d^\star$ with a deliberately *relaxed* matcher—bag-of-content-token containment rather than contiguous substring (§4)—which trades precision for recall and so expands the groundable population. Groundability rises from 35.4% to 43.3% and the retrieved-gold rate from 21.3% to 24.4% (a sixth more questions). If the early-stabilization result were largely a selection artifact, admitting these harder-to-ground questions should pull ϕ_{suf} up sharply. It does not: mean ϕ_{suf} moves only from 0.26

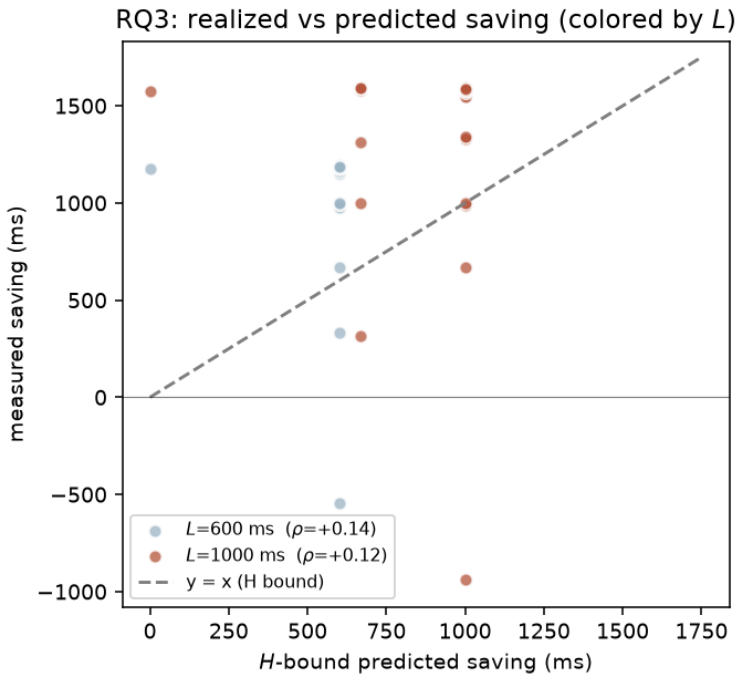

Figure 4: RQ3: per-question measured saving vs. the H bound for the same 60 questions, colored by tool latency L. Most points lie above $y{=}x$ because the pipeline also hides query-generation latency, which H does not model. At $L{=}600$ ms (light) the bound saturates—59/60 points share $H{=}600$ ms; at $L{=}1000$ ms (dark) H spreads across $\{0, 667, 1000\}$ ms, yet measured savings still do not track it (Spearman $\rho = +0.12$). The lone point below zero is a late-stabilizing query whose mis-fire the floored bound cannot express—and whose H is nonetheless maximal.

to 0.281 (median 0.143 → 0.154). The true value is thus bracketed in $[0.26, 0.281]$—both early—and the by-type ordering is preserved, now on firmer footing: `set`, the latest type, grows from $n{=}2$ to $n{=}20$ and remains latest. We read the early-sufficiency finding as signal that survives a 1.5× population expansion, not as an artifact of the strict matcher. (Fuzzy grounding is lower-precision, so it if anything biases ϕ_{suf} *down*; that the estimate barely moves is the reassuring direction.)

Retriever (top-k). Varying the sufficiency top-k shifts absolute values but preserves the qualitative picture (Table 3). Larger k surfaces the gold passage for more questions (retrieved-gold rate 15.6% → 25.0%) and stabilizes slightly earlier (mean ϕ_{suf} 0.327 → 0.248), as expected since a more permissive top-k is easier to satisfy. The self-consistency curve is unchanged by k (it depends only on the top-1), and the central streamable fraction is stable near 73–74%. The early-sufficiency / late-self-consistency gap holds at every k.

6 Threats to Validity

Input timing. A uniform word stream is a proxy for speech; real ASR pacing is variable and partial hypotheses get revised. We bound this here by using clean text and treat speech timing as planned follow-up.

Table 3: Top-k robustness. ϕ_{sc} (mean 0.67 / median 0.73) is independent of k and omitted.

k	retrieved-gold	mean ϕ_{suf}	median ϕ_{suf}
1	15.6%	0.327	0.200
3	21.3%	0.264	0.143
5	25.0%	0.248	0.125

Stabilization measure. Top-1 equality is a coarse proxy for "intent settled." We mitigate by also reporting sufficiency against the gold document and by varying top-k.

Grounding. $d^\star$ is derived by string matching, so the groundable population excludes non-verbatim answers; reported ϕ_{suf} statistics are conditional on groundability, and we report t_{sc} as a grounding-free check.

Retriever and benchmark. BM25 is lexical and phrasing-sensitive, and results are from a single benchmark; a dense-retriever condition and a second QA set are natural extensions.

7 Conclusion

Tool-intent stabilization is a query-intrinsic, model-agnostic quantity that bounds when streaming tool use can help and by how much. On CRAG, sufficiency stabilizes early on the groundable, retrievable subset and is ordered by question type (Kruskal–Wallis $p = 0.017$); the H bound is conservative, with realized savings on a working pipeline meeting or exceeding it; and the streamable fraction is high at realistic operating points. Two limitations temper these claims, and we probed both rather than merely noting them. First, the sufficiency results are conditional on a favorable 21.3% slice; a relaxed-grounding arm that expands the population by a sixth moves ϕ_{suf} by under 0.02 (§5.5), so the early-stabilization finding is bounded as signal, not artifact—though a fuller recovery of the excluded population (dense or learned grounding) remains open. Second, our system validation establishes conservativeness in aggregate but *not* per-query prediction: replaying the same questions at L=1000 ms, where the bound is unsaturated, realized savings still do not track H (Spearman $\rho = +0.12$), because a constant query-generation overlap dominates the saving and trigger mis-fires produce negative outcomes the floored bound cannot represent (§5.4). The downside is real but, measured on both the favorable and the late-stabilizing fallback population, rare ($\approx 1.7\%$ net-negative at L=600). Closing the prediction gap—a model that folds in the query-generation overlap and a mis-fire penalty—together with a dense-retriever condition (H4), a cross-linguistic SOV condition (H5) testing whether the stabilization cue shifts from word order to case morphology, and a regression of ϕ on query features are the natural next steps. Even so, the present results give system builders a principled, training-free basis for trigger design and a build/skip decision for a learned trigger.

References

[1] Gerry T. M. Altmann and Yuki Kamide. Incremental interpretation at verbs: Restricting the domain of subsequent reference. *Cognition*, 73(3):247–264, 1999.

[2] Siddhant Arora, Hamza Khan, Kaihang Sun, Xin Luna Dong, Sneha Choudhary, Seungwhan Moon, Xinyao Zhang, Aman Sagar, Sai Teja Appini, Kushal Patnaik, Shubham Sharma, Shinji Watanabe, Anuj Kumar, Ahmed Aly, Yuan Liu, Florian Metze, and Zhaojiang Lin. Stream RAG: Instant and accurate spoken dialogue systems with streaming tool usage. *arXiv preprint arXiv:2510.02044*, 2025.

[3] David DeVault, Kenji Sagae, and David Traum. Incremental interpretation and prediction of utterance meaning for interactive dialogue. *Dialogue & Discourse*, 2(1):143–170, 2011.

[4] Alvin Grissom II, He He, Jordan Boyd-Graber, John Morgan, and Hal Daumé III. Don't until the final verb wait: Reinforcement learning for simultaneous machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1342–1352, 2014.

[5] Jiatao Gu, Graham Neubig, Kyunghyun Cho, and Victor O.K. Li. Learning to translate in real-time with neural machine translation. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2017.

[6] Yuki Kamide and Don C. Mitchell. Incremental pre-head attachment in japanese parsing. *Language and Cognitive Processes*, 14(5–6):631–662, 1999.

[7] Yuki Kamide, Gerry T. M. Altmann, and Sarah L. Haywood. The time-course of prediction in incremental sentence processing: Evidence from anticipatory eye movements. *Journal of Memory and Language*, 49(1):133–156, 2003.

[8] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, et al. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[9] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[11] Mingbo Ma, Liang Huang, Hao Xiong, Renjie Zheng, Kaibo Liu, Baigong Zheng, Chuanqiang Zhang, Zhongjun He, Hairong Liu, Xing Li, Hua Wu, and Haifeng Wang. STACL: Simultaneous translation with implicit anticipation and controllable latency using prefix-to-prefix framework. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[12] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[13] David Schlangen and Gabriel Skantze. A general, abstract model of incremental dialogue processing. *Dialogue & Discourse*, 2(1):83–111, 2011.

[14] Xiao Yang, Kai Sun, et al. CRAG: Comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.